

WEEK 3 SUMMARY

REST APIs, Entity Framework Core, and SQL Server

Program: Backend Development Internship (.NET)

Week: Week 3 of 10 | Phase 2: Core Technical Training

Theme: REST API design, database persistence, CRUD, and API testing

Duration: 40 hours across five training days

Executive Summary

Week 3 moved the ASP.NET Core Web API from hardcoded in-memory data to a real, persistent SQL Server database. The work combined sound REST API design with normalized relational modeling, Entity Framework Core configuration, code-first migrations, complete asynchronous CRUD endpoints, and end-to-end testing in Postman.

The week established a reusable foundation for later security, testing, deployment, and capstone work. The core principle was consistency: consistent resource-oriented routes, status codes, normalized data, async database access, error handling, and repeatable API tests.

Learning Objectives Achieved

- Applied REST conventions for resource naming, HTTP verbs, response codes, and API versioning.
- Designed a normalized SQL Server schema through third normal form (3NF).
- Configured Entity Framework Core with SQL Server and performed code-first migrations.
- Implemented full asynchronous CRUD operations with EF Core.
- Returned correct responses for validation and not-found scenarios.
- Built a Postman collection that covers both successful and failure paths.

Five-Day Curriculum and Practical Work

Day	Focus	Practical Outcome
1	REST API Design	Resource map, REST conventions, status codes, and versioning.
2	Schema Design	Normalized 3NF schema, keys, relationships, column types, and ERD.
3	EF Core & Migrations	Entity classes, DbContext, SQL Server

Day	Focus	Practical Outcome
		connection, and code-first migration.
4	Async CRUD	Create, read, update, delete, change tracking, and error handling.
5	Postman & Documentation	Organized collection, environments, automated tests, and API documentation.

Day 1: REST API Design and Resource Modeling

REST was presented as an architectural style that models the system around resources such as users, customers, and orders. Resources are represented by plural nouns and operated on with a small, predictable set of HTTP verbs. REST APIs are stateless: every request contains the information needed to process it, which makes horizontal scaling easier.

Endpoint conventions: Use URLs such as GET /api/orders, GET /api/orders/{id}, POST /api/orders, PUT /api/orders/{id}, DELETE /api/orders/{id}, and nested routes such as GET /api/customers/{id}/orders. Verbs belong in HTTP methods, not URL names.

Status-code discipline: Return 200 for successful GET/PUT, 201 Created plus a Location header for successful POST, 204 for a successful DELETE, 400 for invalid input, and 404 when a resource is absent. A 200 response containing an error flag is not a correct substitute for an HTTP error status.

The day also required an early versioning decision, commonly /api/v1/... or a request header. Setting a versioning convention before clients depend on the API avoids disruptive changes later.

Day 2: SQL Server Schema Design and Normalization

Database design began with entities, attributes, and relationships, then applied normalization so each fact is stored once in the appropriate table. A normalized schema avoids duplicated data and update anomalies, such as storing the same customer address on every order.

- 1NF: each column contains an atomic value; do not place lists in a single field.
- 2NF: non-key columns depend on the entire primary key, especially when composite keys are used.
- 3NF: non-key columns depend only on the key, not on other non-key columns.
- Primary keys uniquely identify records; foreign keys enforce valid relationships; many-to-many relationships require a join table.
- Use appropriate column types: decimal for money, not float; bounded string types for short values; and the narrowest type that fits the data.

Example schema: Customers, Orders, Products, and OrderItems form a clear model: Orders references Customers, while OrderItems joins Orders and Products.

Day 3: Entity Framework Core and Code-First Migrations

Entity Framework Core connects C# domain entities to SQL Server. Entity classes map properties to columns, `DbSet<T>` properties represent tables, and `DbContext` tracks changes and translates LINQ queries into SQL. Development can use LocalDB or a SQL Server Docker container.

Code-first migrations keep the C# model and database schema synchronized. The workflow is to create a migration with `dotnet ef migrations add InitialCreate`, inspect the generated change set, then apply it using `dotnet ef database update`. Every future schema change should follow this migration process so database evolution is documented and version controlled.

Secure configuration: Connection strings with real credentials are secrets. Store local values in a gitignored `appsettings.Development.json` and provide production secrets through environment variables or platform secret management, never source control.

Day 4: Asynchronous CRUD Operations

CRUD endpoints were implemented against the actual database. Create adds an entity and calls `SaveChangesAsync`, returning 201 Created with `CreatedAtAction`. Read uses asynchronous LINQ methods such as `ToListAsync` and `FirstOrDefaultAsync`. Update fetches a tracked entity, changes its properties, and calls `SaveChangesAsync`; Delete fetches, removes, and saves.

- Check for a missing entity before update or delete and return 404 Not Found instead of allowing a null-reference failure.
- Validate request data before database work and return a clear 400 Bad Request for missing or out-of-range values.
- Always await `SaveChangesAsync` and every async EF Core operation; ignoring an unawaited-task warning causes difficult timing bugs.
- Use `AsNoTracking` for read-only queries when appropriate to reduce change-tracking overhead on read-heavy endpoints.

Day 5: Postman Testing and API Documentation

Postman was used to create a complete, shareable API collection rather than ad hoc requests. Requests are organized by resource, and every endpoint must include at least one happy-path case and one realistic error-path case. Test scripts can automatically assert a status code and expected response properties, turning manual checks into repeatable verification.

Postman environments store values such as `{{baseUrl}}`, enabling one collection to target local, staging, or production systems without editing each request. The API should also be documented with each endpoint's purpose, required fields, and possible error responses so a teammate can use it without reading the source code.

Required Deliverables

- REST resource design document with endpoints, status codes, and versioning convention.
- Normalized 3NF database schema with an ERD.

- EF Core project with entities, DbContext, SQL Server configuration, and applied migrations.
- Full CRUD implementation for the primary resource against SQL Server.
- Postman collection covering happy and error paths with environments and variables.
- Week 3 summary ready for mentor check-in.

Testing Checklist

1. Verify each endpoint uses the expected HTTP verb, route pattern, and success response.
2. Test GET-by-id, UPDATE, and DELETE with a missing ID and confirm 404.
3. Test CREATE and UPDATE with invalid input and confirm a clear 400 response.
4. Run migrations and inspect the resulting tables, keys, and relationships in a database tool.
5. Use a Postman environment and test scripts to validate at least three expected responses automatically.